



Matheuristic fixed set search applied to the multidimensional knapsack problem and the knapsack problem with forfeit sets

Raka Jovanovic¹ · Stefan Voß^{2,3}

Received: 9 July 2023 / Accepted: 4 January 2024 / Published online: 12 February 2024
© The Author(s) 2024

Abstract

In this paper, we present a solution method for the multidimensional knapsack problem (MKP) and the knapsack problem with forfeit sets (KPFS) using a population-based matheuristic approach. Specifically, the learning mechanism of the fixed set search (FSS) metaheuristic is combined with the use of integer programming for solving subproblems. This is achieved by introducing a new ground set of elements that can be used for both the MKP and the KPFS that aim to maximize the information provided by the fixed set. The method for creating fixed sets is also adjusted to enhance the diversity of generated solutions. Compared to state-of-the-art methods for the MKP and the KPFS, the proposed approach offers an implementation that can be easily extended to other variants of the knapsack problem. Computational experiments indicate that the matheuristic FSS is highly competitive to best-performing methods from the literature. The proposed approach is robust in the sense of having a good performance for a wide range of parameter values of the method.

Keywords Metaheuristics · Multidimensional knapsack problem · Knapsack problem with forfeit sets · Fixed set search · Matheuristic

✉ Raka Jovanovic
rjovanovic@hbku.edu.qa

Stefan Voß
stefan.voss@uni-hamburg.de

¹ Qatar Environment and Energy Research Institute, Hamad bin Khalifa University,
PO Box 5825 Doha, Qatar

² Institute of Information Systems, University of Hamburg, Von-Melle-Park 5, 20146 Hamburg,
Germany

³ Escuela de Ingeniería Industrial, Pontificia Universidad Católica de Valparaíso, Valparaíso,
Chile

1 Introduction

The knapsack problem (KP) and its variants are among the most researched combinatorial optimization problems. Due to the NP-hardness of these problems, many heuristic and metaheuristic methods have been developed for solving them. In this paper, a general method is proposed that can be potentially applied to a wide range of them. The approach is illustrated on two variants of the knapsack problem with different properties. To be specific, the proposed approach is applied on the multi-dimensional knapsack problem (MKP) and the knapsack problem with forfeit sets (KPFS).

The MKP is a versatile problem with a wide range of practical applications such as cutting stock (Gilmore and Gomory 1966), loading (Shih 1979), and many others. However, solving it is computationally challenging due to its NP-hardness (Garey and Johnson 1979). Despite this, it has been extensively studied, and numerous solution approaches have been proposed. A comprehensive review of representative studies up to 2004 can be found in Fréville (2004), and more recent studies are discussed in Lai et al. (2018). Interesting variants of the MKP include the multiple multidimensional knapsack problem (Mancini et al. 2021), the multiple-choice multidimensional knapsack problem (Chen and Hao 2014), the robust multiple-choice multidimensional knapsack problem (Caserta and Voß 2019), and the multidemand multidimensional knapsack problem (Lai et al. 2019).

Another group of variations of the KP is based on the idea of incompatible items (Basnet 2018; Coniglio et al. 2021). One of the main representatives of this type of problem is the knapsack problem with conflict graphs (KPCG), which deals with incompatibilities between item pairs and is proven to be strongly NP-hard (Pferschy and Schauer 2009; Li et al. 2021). A closely related variant is the knapsack problem with forfeits (KPF), where item pairs come with associated penalty costs (Capobianco et al. 2022). In this paper, the analysis focuses on the newly introduced knapsack problem with forfeit sets (KPFS) which can be understood as a generalization of the KPF (D'Ambrosio et al. 2023). In it, forfeit costs are associated with subsets of items, and an allowance parameter determines how many items can be chosen from each set before incurring penalty costs.

1.1 Related work

Methods for solving the MKP can be exact or heuristic. Exact ones frequently use branch and bound, like those by Shih (1979) and Vimont et al. (2008), and hybridization with other strategies like Boussier et al. (2010) and Mansini and Speranza (2012). The best exact algorithms (e.g., Vimont et al. (2008), Boussier et al. (2010), and Mansini and Speranza (2012)) produce optimal solutions quickly for small benchmark instances but have a prohibitive computational cost for large ones. Besides exact solution methods, the literature contains several heuristic algorithms categorized as either single-solution-based local search or population-based optimization. Local search algorithms, such as tabu search (Dammeyer and Voß 1993;

Glover and Kochenberger 1996; Hanafi and Freville 1998; Vasquez and Vimont 2005), simulated annealing (Drexler 1988), and kernel search (Angelelli et al. 2010), have been shown to be successful. Population-based methods include genetic and memetic algorithms (Chu and Beasley 1998; Rezoug et al. 2018), hybrid binary particle swarm optimization (Haddar et al. 2016; Mingo López et al. 2018), ant colony optimization (Al-Shihabi and Ólafsson 2010), path relinking (Arin and Rabadi 2016), and many others. The MKP also attracts research related to general artificial intelligence exposition. For instance, in García et al. (2020), the authors propose an improved binarization framework, which uses the K -means technique to enable the use of continuous metaheuristics for the MKP.

One of the best-performing metaheuristic algorithms for the MKP is the two-phase tabu-evolutionary algorithm (TPTEA) (Lai et al. 2018) which was also highly successful for the multidemand multidimensional knapsack problem (Lai et al. 2019). This approach combines two solution-based tabu search methods with an evolutionary framework that utilizes a hyperplane-constrained crossover operator to generate offspring solutions. Additionally, it employs a dynamic method to identify areas of interest for the search and a diversity-based population updating rule to ensure that a diverse and healthy population is maintained. Another effective method for the MKP, known as the diversity-preserving quantum particle swarm optimization (Lai et al. 2020), relies on quantum particle swarm optimization (Haddar et al. 2016) and exhibits a substantially lower computational cost compared to TPTEA. However, it achieves solutions of lower quality, as demonstrated by Lai et al. (2020). This technique combines a diversity-preserving strategy based on distance to manage the population, along with a solution improvement method that employs variable neighborhood descent for local optimization.

Since the KPFS is a newly introduced problem, only limited research has been done on the development of solution methods. To be specific, in the work of D'Ambrosio et al. (2023), a mixed integer programming model has been introduced. In the same paper, a fast solution method based on the carousel greedy algorithm is also presented. In addition, an advanced memetic algorithm (MA) is proposed which extends the genetic algorithm paradigm by including a local refinement mechanism.

1.2 Fixed set search

The fixed set search (FSS) is a population-based metaheuristic that adds a learning mechanism to the greedy randomized adaptive search procedure (GRASP) (Feo and Resende 1995). The GRASP involves generating solutions using a randomized greedy algorithm and applying a local search to each of them. The FSS has been successfully applied to solve several problems, including the traveling salesman problem (Jovanovic et al. 2019), the power dominating set problem (Jovanovic and Voss 2020), machine scheduling (Jovanovic and Voß 2021), the minimum weighted vertex cover problem (Jovanovic and Voß 2019), the covering location with inter-connected facilities problem (Lozano-Osorio et al. 2023), the clique partitioning

problem (Jovanovic et al. 2023b), as well as bi-objective optimization problems (Jovanovic et al. 2022). The FSS is inspired by the fact that high-quality solutions for a specific problem instance often have many of the same elements in common. Therefore, the FSS generates new solutions that contain these fixed elements, and the computational effort is focused on completing the partial solution. The idea of using frequently occurring elements in high-quality solutions is based on earlier notions of chunking (Voß and Gutenschwager 1998; Woodruff 1998), vocabulary building, and consistent chains (Sondergeld and Voß 1999) as they have been used in relation to tabu search. Closely related concepts have also been used in the matheuristic POPMUSIC paradigm (Taillard and Voß 2002).

Matheuristic approaches, such as kernel search (Angelesli et al. 2010; Maniezzo et al. 2021) and heuristic concentration (Rosing and ReVelle 1997), are based on the idea of generating numerous solutions to identify elements that commonly appear in high-quality solutions. These methods then fix these elements and solve the corresponding mathematical programming problem. On the other hand, the FSS utilizes a mechanism that can generate various fixed sets or kernels based on elements that frequently appear in different subsets of high-quality solutions, resulting in a more efficient global search. While kernel search and heuristic concentration share similarities with the FSS, the latter offers a more diverse range of fixed sets or kernels.

1.3 Contributions

One of the drawbacks of the best-performing methods for the MKP and the KPFS is that they have a highly complex implementation. The main reason is that they hybridize different metaheuristics and use several distinct solution neighborhoods. In this work, the goal is to provide a simple-to-implement method to solve the MKP and the KPFS based on the FSS. To be specific, the idea is to combine the FSS with the use of integer programming (IP). Related methods combining heuristics/metaheuristics with IPs are frequently called matheuristics, for which a recent review can be found in Boschetti and Maniezzo (2022). It should be noted that a method of this type has recently been successfully applied to the closely related quadratic multiple knapsack problem (Galli et al. 2023).

In the development of the matheuristic FSS (MFSS) for the MKP and the KPFS, several new concepts are explored in relation to the basic FSS. Firstly, a novel way for defining the ground set of elements for 0-1 problems is introduced that aims to maximize the amount of information that the fixed set provides. Next, an effective mechanism is proposed for incorporating the learning mechanism in a matheuristic setting. In this way, the need for defining a randomized greedy algorithm and a local search, as in the original FSS, can be avoided making the implementation of the method less complex. Another novel idea in the MFSS is using the method for generating fixed sets to diversify the generated solutions. The conducted computational

experiments show that the MFSS is highly competitive to state-of-the-art methods for the MKP and outperforms the ones for the KPFS.

The paper is organized as follows. Section 2 is dedicated to the problem formulations of the MKP and the KPFS. Next, Sect. 3 provides details of applying the MFSS to the problems of interest. The following Sect. 4 is dedicated to the presentation of the conducted computational experiments. The paper is finalized in Sect. 5 with some concluding remarks.

2 Problem formulations

The knapsack problem is defined for a set of items V and a capacity c . Each item $j \in V$ has a non-negative profit value p_j and a non-negative weight w_j . The goal is to select a set of items $S \subset V$ such that the sum of the profit values of items in S is maximized while satisfying the constraint that the sum of the weights is less or equal to the capacity c .

2.1 The multidimensional knapsack problem

The multidimensional knapsack problem (MKP) extends this concept to have multiple constraints. Now, the capacity is an m -dimensional vector having values c_i , for $i = 1..m$. Each item $j \in V$ has an m -dimensional weight vector, having as value w_{ij} . The goal is to maximize the sum of profit values while ensuring that the sum of the weights in each dimension $i = 1..m$ does not exceed the capacity c_i .

Formally, the MKP can be specified using an IP model with a set of binary decision variables x_j defined for $j \in V$ using the following objective function:

$$\text{Maximize} \quad \sum_{j \in V} p_j x_j \quad (1)$$

Subject to

$$\sum_{j \in V} w_{ij} x_j \leq c_i \quad i = 1..m \quad (2)$$

$$x_j \in \{0, 1\} \quad j \in V \quad (3)$$

Constraints (2) guarantee the satisfaction of the capacity constraints for each of the m dimensions, and variable definitions are provided in (3).

2.2 The knapsack problem with forfeit sets

In this subsection, the formulation of the KPFS is given as proposed by D'Ambrosio et al. (2023). The KPFS uses the same set of items $j \in V$, item profit values p_j ,

item weights w_j , and capacity c as in the KP. Also, a solution $S \subseteq V$ must satisfy the constraint that the total weight of the items in the solution is less or equal to the knapsack capacity c .

Additionally, there exists a collection of l forfeit sets, denoted as $C = \{C_1, \dots, C_l\}$, where each set C_i is a subset of V . These sets satisfy the condition $|C_i| \geq 2$ for $i = 1, \dots, l$, where the notation $|C_i|$ is used for the cardinality of the set. Each set C_i is associated with a non-negative cost d_i and an integer allowance h_i , ensuring $0 \leq h_i \leq |C_i|$. For a given solution S , $n_i^S = |C_i \cap S|$ is introduced to represent the number of elements shared between C_i and S . If $n_i^S > h_i$, a penalty equal to $(n_i^S - h_i)d_i$ must be paid. In such instances, we state that $n_i^S - h_i$ violations are linked to C_i in solution S . Finally, an integer upper bound $k \geq 0$ is imposed on the total number of violations allowed in a solution. Formally, a solution S is considered feasible if and only if:

$$\sum_{i \in \{1, \dots, l\} : n_i^S > h_i} (n_i^S - h_i) \leq k \quad (4)$$

This condition ensures that the cumulative violations across all forfeit sets in S do not exceed the predefined limit k .

The integer programming formulation is as follows. For each $j \in V$, a binary variable x_j is defined and equals 1 if j is chosen, and 0 otherwise. Next, for each $C_i \in C$, the integer variable v_i is defined and represents the number of violations associated with the set. Using these decision variables, the IP for the KPFS can be fully specified using the following formulation.

$$\text{Maximize} \quad \sum_{j=1}^n p_j x_j - \sum_{i=1}^l d_i v_i \quad (5)$$

Subject to

$$\sum_{j=1}^n w_j x_j \leq c \quad (6)$$

$$\sum_{i=1}^l v_i \leq k \quad (7)$$

$$\sum_{j \in C^i} x_j - v_i \leq h_i \quad i = 1..l \quad (8)$$

$$x_j \in \{0, 1\} \quad j \in V \quad (9)$$

$$v_i \in \{0, \dots, |C^i| - h_i\} \quad i = 1..l \quad (10)$$

The objective function, given in (5), maximizes profit, which is the sum of the profits of all selected items minus the associated costs. Constraint (6) ensures that the sum of the weights of items in the solution does not exceed the capacity. Similarly, Constraint (7) enforces the limit on the number of allowed violations. Constraints (8) represent the relationship among the selected items, the allowance value, and the resulting violations for each forfeit set. Finally, Constraints (9)–(10) define the domain of the decision variables. It should be noted that, as proven by D'Ambrosio et al. (2023), in the formulation of the KPFS, it is not necessary for variables v_i to be integer.

3 A matheuristic based on the fixed set search

The FSS algorithm takes advantage of the fact that many high-quality solutions for a particular combinatorial optimization problem have common elements. The FSS incorporates some of these elements into newly generated solutions and focuses computational effort on finding optimal or near-optimal solutions in the corresponding subset of the solution space. This selected set of common elements is called the “fixed set.” The goal of the FSS is to “fill in the gaps” and complete the partial solution corresponding to the fixed set. In the FSS, this is exploited through adding a learning mechanism to the GRASP. In this section, we present the MFSS which extends this concept to a matheuristic setting. The MFSS consists of several building blocks, including representing the problem solution as a subset of a ground set of elements, defining methods for generating fixed sets, implementing the learning mechanism, and defining a method for completing a solution from a fixed set.

3.1 Fixed set

In this section, we describe the approach for generating fixed sets for the MKP and the KPFS. To use the MFSS algorithm, a solution must be represented as a subset of a ground set of elements. For the two problems of interest, a natural way to represent the solution is as a subset S of the set of all items V . In the adaptation of the FSS to a matheuristic approach, our goal is to use a representation of a solution that provides as much information as possible on all items $i \in V$. Because of this, an alternative ground set is used. Let us note that each item $i \in V$ can either be inside the knapsack or outside of it. Using this idea, the ground set can be defined as $G = V \times \{T, \perp\}$, where a pair (i, T) means that item i is selected to be inside the knapsack and $(i, \perp)$ the opposite. Now, a solution S is a subset of the ground set G and satisfies $|S| = |V|$, or in other words has the same cardinality as the set of items. A graphical illustration of the two types of ground sets is shown in Fig. 1.

The next step is defining a procedure for generating multiple fixed sets F with a controllable size (cardinality) $|F|$. Note that it should be possible to use such fixed sets to produce feasible solutions of equal or higher quality than the solutions already

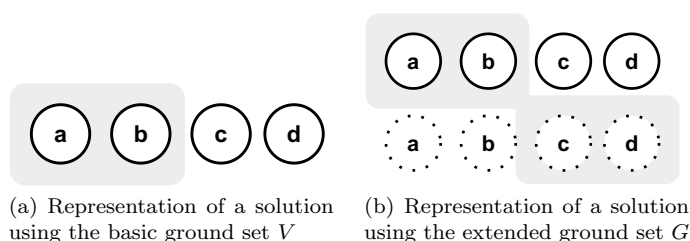


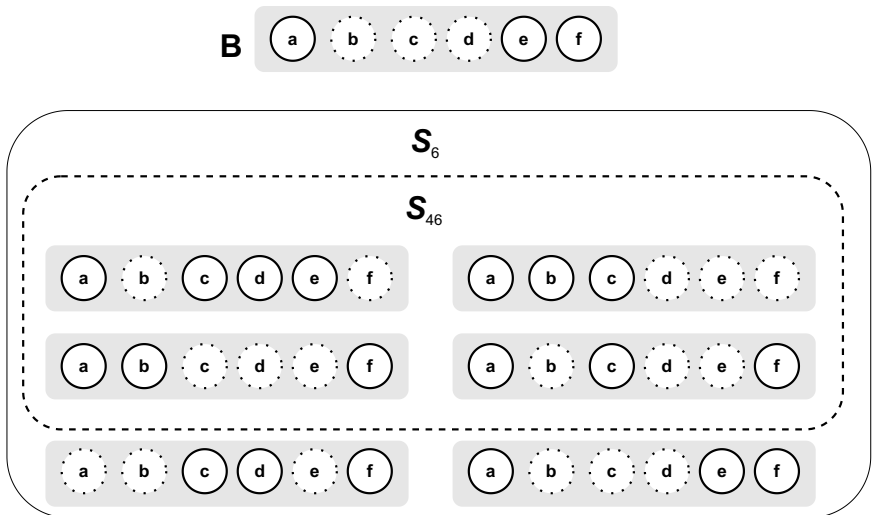
Fig. 1 Illustration of the different ground sets for the MKP and the KPFS. The items inside the gray shape represent the solution. In case of the extended ground set, circles with full lines indicate that an item is inside the knapsack and a circle with a dashed line the opposite

generated. We begin with some definitions: The notation $\mathcal{S}_n = \{S_1, \dots, S_n\}$ represents the set of n best solutions generated in the previous steps of the algorithm. A base solution $B \in \mathcal{S}_n$ is a randomly selected solution from the best n solutions. If the fixed set satisfies $F \subset B$, it can be used to generate a feasible solution of at least the same quality as B . Moreover, F can contain an arbitrary number of elements of B . The idea is to create F such that it contains frequently occurring elements in a group of high-quality solutions. We define $\mathcal{S}_{kn}$ as the set of k randomly selected solutions out of the n best solutions, $\mathcal{S}_n$. Let us define the function $C(e, S)$, for a solution S and element e , as 1 if $e \in S$ and 0 otherwise. Using $C(e, S)$, we can count that the number of times an element e occurs in $\mathcal{S}_{kn}$ with the function:

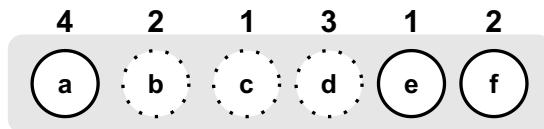
$$O(e, \mathcal{S}_{kn}) = \sum_{S \in \mathcal{S}_{kn}} C(e, S) \quad (11)$$

Then, we define $F \subset B$ as the set of elements e with the largest value of $O(e, \mathcal{S}_{kn})$. Furthermore, we define the function $F = \text{Fix}(B, \mathcal{S}_{kn}, \text{Size})$ as the fixed set generated for a base solution B , a set of solutions $\mathcal{S}_{kn}$ with Size elements.

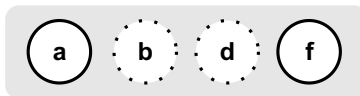
In the case of the original FSS, the diversification is done through the use of a randomized greedy algorithm with a pre-selected set of elements. In the case of the MFSS, this is partly done in the method for generating the fixed set. To be more precise, we utilize the way ties are resolved. Let us make the following observation. The last element e that should be added to the fixed set F has the value of $O(e, \mathcal{S}_{kn}) = f$. In the general case, there are multiple elements that have the same value of this function. Let us assume that there is a total of $l > \text{Size}$ elements $e \in B$ that have a value of the function $O(e, \mathcal{S}_{kn})$ greater or equal than f . Let us use the notation $\hat{F}$ for the set of such elements. In the proposed approach, in such cases, the function $\text{Fix}(B, \mathcal{S}_{kn}, \text{Size})$ returns $\hat{F}$ with $l - \text{Size}$ random elements removed. Note that a removed element e does not necessarily have the lowest value of $O(e, \mathcal{S}_{kn})$. A graphical illustration of the method for generating fixed sets is shown in Fig. 2.



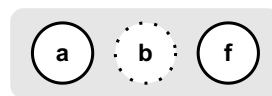
(a) The inputs for the *Fix* method are the base solution B and the set S_{46} test solutions



(b) The values of the function O for all the elements of the base solution B



(c) The set $\hat{F}$ with 4 elements



(d) The set F is equal to $\hat{F}$ with a random element removed

Fig. 2 Illustration of the method for generating fixed sets. It is assumed that the size of the fixed set is $Size = 3$. Next, the number of used best solutions is 6, and four test solutions are used. An element of a solution is represented using circles with full or dashed lines indicating if the item is inside or outside the knapsack. A solution is represented using the elements inside a gray rectangle

3.2 Integer program use

The idea of the proposed matheuristic approach is to exploit the fact that for many problems, the use of an IP solver is highly efficient up to a specific instance size. Moreover, standard IP solvers, in the general case, for such instance sizes manage to find a near-optimal solution at a low computational cost, and the majority of the time is often spent on finding the optimal solution and proving its optimality.

In the proposed work, the idea is to use fixed sets to create an IP that can be efficiently solved using standard solvers. A simple way to decrease the computational cost of solving an IP is by fixing the values of some decision variables. This can naturally be done using fixed sets by adding the following set of constraints to the IP model.

$$x_i = 0 \quad i \in \{j \mid (j, \perp) \in F\} \quad (12)$$

$$x_i = 1 \quad i \in \{j \mid (j, \top) \in F\} \quad (13)$$

Equation (12) guarantees that for any element of the fixed set F of the form $(j, \perp)$, item j is not inside the knapsack. Analogously, for an element $(j, \top)$, j is in the knapsack. Note that for the MKP, this model is equivalent to solving the knapsack problem with all the items that do not appear in the fixed set with appropriately adapted capacity constraints.

Let us define $IPS(F, \tau)$ as a function for solving the IP defined by using the IP formulation of the original problem with the added constraints (12) and (13) with a maximal computational time of τ . Note that in the case of the MKP, the formulation of $IPS(F, \tau)$ consists of (1)–(3) and constraints given in (12) and (13), while for the KPFS, the formulation consists of (5)–(10) and constraints given in (12) and (13).

In the practical use of IP solvers, there is often a significant advantage in providing an initial, high quality, incumbent solution S for a “warm start.” This makes it possible to eliminate portions of the search space in the branch-and-cut algorithm and thus may result in smaller branch-and-cut trees. In the proposed method, this is exploited to enhance the performance of the IP solver. Let us define the function $IPS(F, \tau, S)$ that extends the function IPS by providing an initial incumbent solution S . In relation, let us define a function $\text{BestFit}(S, F)$ for a set of solutions S and a fixed set F that is equal to the highest quality solution $S \in S$ that can be acquired by extending the fixed set F as follows.

$$\text{Fit}(S, F) = \{S \mid S \in \mathcal{S} \wedge F \subset S\} \quad (14)$$

$$\text{BestFit}(S, F) = \arg \max_{S \in \text{Fit}(S, F)} z(S) \quad (15)$$

Equation (14) defines the function $\text{Fit}(S, F)$ that returns all the solutions $S \in \mathcal{S}$ for which $F \subset S$. Equation (15) returns the solution that has the highest value of the objective function. Here, the notation $z(S)$ is used for the objective value corresponding to solution S .

3.3 Learning mechanism

Let us give an outline of the MFSS. The idea is to start with a population $\mathcal{P} = \{S_1, \dots, S_{|\mathcal{P}|}\}$ of randomly generated solutions. The next step consists of

iteratively improving the solutions in $\mathcal{P}$ using the following procedure. A random base solution B is selected from the set of n best solutions, S_n . In addition, the set S_{kn} is generated by selecting k solutions from S_n . A fixed set F is generated using B and S_{kn} , and a new solution S' is generated using the IP with the additional constraints related to the fixed set F , as described in the previous subsection. The solution S' is added to the population of solutions $\mathcal{P}$. Note that $\mathcal{P}$ is a set of solutions, so it does not contain duplicates. The procedure is repeated for the new population of solutions. In the further text details about the implementation of the proposed method are provided.

Firstly, a random solution S can be generated using the following simple procedure. Start with an empty solution $S = \emptyset$, and iteratively add items $t \in V$, one by one in a random order, to S if the new solution does not violate any of the capacity constraints and increases the value of the objective function.

In the iterative process, integral to the implementation of the learning mechanism, various considerations come into play. Let us first delve into the aspect of computational effort, given that the computational cost of the IPS can potentially be high. One issue is that, for a low-quality fixed set, even with a prolonged execution time granted to the IPS, it is improbable that high-quality solutions will be attained. Conversely, for a high-quality fixed set, it is reasonable to extend the IPS's execution time, as it explores a part of the solution space containing high-quality solutions.

It is evident that the quality of solutions in the population increases as more solutions are generated. Consequently, the quality of the fixed sets also improves over time. As a result, it is reasonable to allocate shorter computational times for the IPS in the initial iterations of the algorithm and extend them in the later iterations. The proposed method starts with an initial allowed computational time, τ , for the IPS, and increases it when the algorithm shows signs of stagnation. To be more precise, the algorithm is deemed stagnant if, in the last *StagMax* iterations, no new solution within the best n solutions is generated.

Another thing that should be considered is that when the value of allowed computational time τ is above some threshold, the IPS is consistently solved to proven optimality. The two main reasons for this are that the initial solution for the IPS is of very high quality or that the problem is highly constrained. In such cases, when stagnation occurs, it is reasonable to decrease the size of the fixed set $|F|$ that will be used in further iterations since an additional increase in the allowed computational time τ would have no effect, and in this way, a larger neighborhood of the base solution is explored.

Algorithm 1 Pseudocode for MFSS

```

1: Input: Initial population size  $N_{pop}$ , number of best solutions from which the base and
   test solutions are selected  $n$ , subproblem size  $Size_{max}$ , initial computational time for
   the IP  $t_{init}$ , the portion of acquired proven optimal solutions for which the size of the
   fixed set is decreased  $\delta$ , the coefficient for decreasing the fixed set size  $\gamma$ 
2: Output: The best-found solution  $S_{best}$ 
3: function  $MFSS(N_{pop}, n, Size_{max}, t_{init}, \gamma, \delta)$ 
4:    $\tau \leftarrow t_{init}$ 
5:    $Stag \leftarrow 0$ 
6:    $OptCounter \leftarrow 0$ 
7:    $SolveCounter \leftarrow 0$ 
8:    $S_{best} \leftarrow 0$ 
9:    $Size = |V| - Size_{max}$ 
10:  Generate initial population  $\mathcal{P}$ 
11:  while Not Time Limit Reached do
12:    Select random  $k \in [k_{min}, k_{max}]$ 
13:    Select random  $B \in S_n$ 
14:    Generate random  $S_{kn}$ 
15:     $F \leftarrow Fix(B, S_{kn}, Size)$ 
16:     $S_{start} \leftarrow BestFit(\mathcal{P}, F)$ 
17:     $S \leftarrow IPS(F, \tau, S_{start})$ 
18:     $SolveCounter \leftarrow SolveCounter + 1$ 
19:    if  $IPS(F, \tau, S_{start})$  solved to optimality then
20:       $OptCounter \leftarrow OptCounter + 1$ 
21:    end if
22:    if  $z(S) < z(S_{best})$  then
23:       $S_{best} \leftarrow S$ 
24:    end if
25:    if  $S \notin \mathcal{P} \wedge z(S) \geq z(S_n)$  then  $\triangleright S_n$  is the  $n$ -th best solution
26:       $Stag = 0$ 
27:    else
28:       $Stag = Stag + 1$ 
29:    end if
30:    if  $Stag \geq MaxStag$  then
31:      if  $OptCounter \geq \delta SolveCounter$  then
32:         $Size_{max} \leftarrow \gamma Size_{max}$ 
33:         $Size \leftarrow |V| - Size_{max}$ 
34:      else
35:         $\tau \leftarrow 2\tau$ 
36:      end if
37:       $Stag = 0$ 
38:       $SolveCounter \leftarrow 0$ 
39:       $OptCounter \leftarrow 0$ 
40:    end if
41:     $\mathcal{P} = \mathcal{P} \cup \{S\}$ 
42:  end while
43: end function

```

One can obtain a better comprehension of this approach by observing the pseudocode presented in Alg. 1. The first step is setting the size of the fixed set $Size$ that corresponds to the subproblem that can be efficiently solved using the IPS. It is equal to the total number of items $|V|$ minus the maximal number of items $Size_{max}$ for which the IPS is effective. Next, the procedure generates an initial population of N_{pop} solutions using a previously defined method for creating feasible solutions. Within the main

loop, the first step involves randomly selecting an integer value k for the size of the set $\mathcal{S}_{kn}$, within the interval $[k_{\min}, k_{\max}]$. To increase the diversity of generated fixed sets, the value of k changes in subsequent iterations. Next, the random base solution B and the set $\mathcal{S}_{kn}$ are chosen from the set of the n best solutions. Subsequently, a fixed set F is generated by applying the function $Fix(B, \mathcal{S}_{kn}, \text{Size})$. The initial incumbent solution S_{start} is selected as the solution with the best quality that fits the fixed set F from the population of solutions $\mathcal{P}$. The next step involves using the function $IPS(F, \tau, S_{\text{start}})$, with a time limit τ and the initially incumbent solution S_{start} , to obtain a new solution S . Next, it is evaluated if S is the new best solution. After that, it is checked if S will be added to the set $\mathcal{S}_n$. If this is not the case, the stagnation counter is increased by 1; otherwise, it is set to 0. After this step, the procedure verifies whether stagnation has occurred, and if so, and proven optimal solutions have been acquired above a certain threshold ($OptCounter \geq \delta SolveCounter$, where δ is a method parameter) since the last stagnation, the size of the fixed set is decreased based on the coefficient γ . Otherwise, if stagnation has occurred, the allowed computational time τ for IPS is doubled. Finally, the new solution S is added to the population of solutions $\mathcal{P}$. This process is repeated until the time limit is reached.

4 Results

In this section, the results of the conducted computational experiments are presented. The first objective is to evaluate the effectiveness of the proposed MFSS. This is done in comparison with state-of-the-art metaheuristic methods. The second goal is to assess the impact of the method parameters. The MFSS approach has been implemented in ILOG CPLEX for C#.NET through Concert Technology. The computational experiments have been performed on a personal computer running Windows 10 having Intel(R)Xeon(R) Gold 6244 CPU @3.60 GHz with 128 GB memory.

This section is organized as follows. The next two subsections focus on the evaluation of the performance of the MFSS for the MKP and the KPFS. The last subsection is dedicated to the evaluation of the effect of the parameters of the MFSS.

4.1 Evaluation of the MFSS for the MKP

In this subsection, the computational experiments used to evaluate the MFSS for the MKP are presented. In the following text, first, the properties of the used test instances are given, and later, a detailed comparison to state-of-the-art methods is provided.

4.1.1 Test instances

The computational experiments are carried out on the standard benchmark instances that are a part of the OR-Library for which a detailed description can be found in Chu and Beasley (1998). Only the results for the medium and large instances are presented since all the best-performing methods find optimal solutions for small

instances. The instances can be categorized into two sets with distinct characteristics, described as follows:

- **Medium-sized instances:** This set comprises 90 instances with $|V| = 250$ items, divided into three subsets. Each subset contains 30 instances with the number of constraints m equal to 5, 10, or 30. The coefficients w_{ij} ($i = 1..m$, $j = 1..|V|$) are randomly generated integers in the range $[0, 1000]$. The value of c_i is set to $\beta \times \sum_{j=1..|V|} w_{ij}$ for $i = 1..m$, where β is a parameter known as the tightness ratio. For each pair $|V|$ and m , three groups of 10 instances are generated. Each group uses a single value of β equal to 0.25, 0.5, or 0.75. The profit coefficients p_j are correlated to the values w_{ij} as follows:

$$p_j = \sum_{i=1}^m \frac{w_{ij}}{m} + 500q_j \quad j = 1..|V| \quad (16)$$

where q_j is a random real number drawn from the interval $(0, 1)$ with a uniform distribution.

- **Large-sized instances:** This set comprises 90 instances with $|V| = 500$ items, divided into three subsets. Each subset contains 30 instances with the number of constraints m equal to 5, 10, or 30. The method for generating the coefficients c_i , p_j and w_{ij} is the same as for medium-sized instances.

Optimal solutions for the instances with $m = 5$ and $m = 10$ are provided in Vimont et al. (2008), Boussier et al. (2010), and Mansini and Speranza (2012). It is important to note that obtaining optimal solutions acquired for instances reported in these references required a significant computation time, with some instances taking up to 150 h for $|V| = 500$ and $m = 10$.

4.1.2 Comparison to other methods

To evaluate the efficacy of the proposed algorithm, we select five state-of-the-art heuristic algorithms as the primary reference algorithms, which have been well documented in the existing literature. These reference algorithms include the genetic algorithm (GA) (Chu and Beasley 1998), which serves as a baseline reference. We also select other well-known algorithms such as the filter-and-fan heuristic (F&F) of Khemakhem et al. (2012), the hybrid quantum particle swarm optimization algorithm (QPSO) of Haddar et al. (2016), along with its variation incorporating diversity preservation (DQPSO) (Lai et al. 2020), and the two-phase tabu-evolutionary algorithm (TPTEA) (Lai et al. 2018). These algorithms are chosen based on their strong performance and status as some of the most effective heuristic algorithms for the MKP currently available in the literature.

The same set of MFSS parameters has been used for all the test instances in the comparison. These values have been found empirically. The size of the initial population size N_{pop} is 100. The maximal size of a subproblem *SizeMax* is 100. The

initial maximal computational time for the IPS is 0.1 s. The number of n best solutions that are considered for selecting the base solution and S_{kn} is 100. The value of the parameter k for generating S_{kn} is a randomly selected integer value from the interval [5, 9]. The algorithm is considered stagnant if in the last $MaxStag = 50$ iterations, there have been no changes of the set of the n best solutions S_n . The maximal allowed calculation time for the MFSS is 600 s. It should be noted that in case of the MKP, the maximal size of the subproblem that has been solved, $SizeMax$, has never been changed. This is due to the fact that proven optimal solutions have not been frequently found; consequently, the MFSS parameters γ and δ have not been used.

The computational experiments are performed on the instances described in the previous subsection. The setting of the experiments is the same as the one used in Lai et al. (2020); for each test instance, ten independent runs are performed. The evaluation is done based on the difference between the optimal or best-known solution for an instance to the solution acquired by the methods used in the comparison. In the later text, this difference is called the error. Since ten independent runs are performed for each method on each instance, aggregated values of these runs are observed. To be specific, the minimal error over all the runs and the average error are used. The results of the comparison are shown in Tables 1, 2, 3, 4, 5, and 6 for instance groups with a different number of items and constraints.¹ Note that the values for the other methods have been taken from the corresponding papers. For GA and F&F, the average results over ten independent runs were not available in the published papers.

The first thing that can be observed is that the TPTEA, DQPSO, and MFSS have significantly better results than the other methods. In case of instances with 250 items with 5 or 10 constraints (Tables 1 and 2) and with 500 items and 5 constraints (Table 4) out of 90 instances, TPTEA, DQPSO, and MFSS missed on finding all optimal solutions only in 2, 9, and 2 cases, respectively. For the last group of instances with known optimal solutions with 500 items and 10 constraints (Table 5), TPTEA, DQPSO, and MFSS find 12, 8, and 9 optimal solutions out of 30 instances, respectively. In summary, the MFSS clearly outperforms DQPSO over all the metrics for all instance groups. Its performance is very similar to that of TPTEA. It has a more robust behavior than TPTEA regarding the average solution quality over 10 runs for each instance being better in five out of six test groups, although the difference is relatively small. On the other hand, when the quality of the best-found solutions is compared, TPTEA and MFSS have a very similar behavior. TPTEA manages to find better solutions for a few more instances than MFSS. On the other hand, the MFSS has better average values of best-found solutions for a few problem sizes. In case of the hardest problem groups, without known optimal solutions in Tables 3 and 6, the MFSS and TPTEA have been once better/worse than the other method when the average quality of the best-found solution or average solution quality over the 10 runs are compared. On the other hand, TPTEA managed to find 29 and 12 best-known solutions out of 30 test instances compared to MFSS's 28 and 7 for the hardest instances with 250 and 500 items, respectively.

¹ Underlined Avg. values indicate best values in the respective comparison.

Table 1 Comparison of the state-of-the-art methods to MFSS on instances with 250 items and five constraints

ID	OPT	Best						Average			
		GA	F&F	QPSO	TPTEA	DQPSO	MFSS	QPSO	TPTEA	DQPSO	MFSS
0	59,312	0	0	0	0	0	0	0.00	0.00	0.00	0.00
1	61,472	0	4	0	0	0	0	2.00	0.00	0.28	0.00
2	62,130	0	0	0	0	0	0	0.00	0.00	0.00	0.00
3	59,463	17	27	36	0	0	0	36.00	0.67	21.07	3.00
4	58,951	0	0	0	0	0	0	0.00	0.00	0.00	0.00
5	60,077	21	15	0	0	0	0	21.00	7.50	6.66	0.00
6	60,414	0	0	0	0	0	0	0.00	0.00	0.00	0.00
7	61,472	0	18	0	0	0	0	11.50	0.00	0.00	0.00
8	61,885	0	0	0	0	0	0	0.00	0.00	0.00	0.00
9	58,959	0	0	0	0	0	0	33.50	0.00	0.00	0.00
10	109,109	0	0	43	0	0	0	50.50	0.00	1.54	0.00
11	109,841	0	0	0	0	0	0	0.00	0.00	0.00	0.00
12	108,508	19	0	0	0	0	0	0.00	0.00	0.00	0.00
13	109,383	0	0	27	0	0	0	35.50	0.00	0.00	0.00
14	110,720	0	0	0	0	0	0	10.00	0.00	1.90	0.00
15	110,256	0	0	0	0	0	0	0.00	0.00	5.29	0.00
16	109,040	24	0	0	0	0	0	17.50	0.00	0.00	0.00
17	109,042	5	26	0	0	0	0	23.50	0.00	0.68	0.00
18	109,971	14	14	0	0	0	0	16.00	0.00	0.00	0.00
19	107,058	20	0	0	0	0	0	10.00	0.00	0.91	0.00
20	149,665	6	6	0	0	0	0	14.50	0.00	3.90	0.00
21	155,944	4	0	0	0	4	0	2.00	0.13	4.00	0.00
22	149,334	18	0	0	0	0	0	0.00	0.00	1.54	0.00
23	152,130	0	0	0	0	0	0	0.00	0.00	0.00	0.00
24	150,353	0	0	0	0	0	0	0.00	0.00	0.00	0.00
25	150,045	0	0	0	0	0	0	0.00	0.00	0.00	0.00
26	148,607	0	0	0	0	0	0	0.00	0.00	0.00	0.00
27	149,782	10	0	10	0	0	0	19.50	0.00	6.60	0.00
28	155,075	0	0	18	0	0	0	30.00	0.00	0.00	0.00
29	154,668	6	0	0	0	0	0	0.00	0.00	0.00	0.00
Avg.		5.47	3.67	4.47	0.00	0.13	0.00	11.10	0.28	1.81	<u>0.10</u>
#worse		12	7	5	0	1		16	2	12	
#equal		18	23	25	30	29		14	27	18	
#better		0	0	0	0	0		0	1	0	

The evaluation is done based on the difference between the optimal solution and the solution acquired by a method ($OPT - X$). The columns "Best" represent the minimal difference, and "Average" is the average difference over 10 independent runs

Table 2 Comparison of the state-of-the-art methods to MFSS on instances with 250 items and 10 constraints

ID	OPT	Best		Average							
		GA	F&F	QPSO	TPTEA	DQPSO	MFSS	QPSO	TPTEA	DQPSO	MFSS
0	59,187	0	23	5	0	0	0	14.00	0.00	0.00	0.00
1	58,781	119	88	0	0	76	0	48.00	37.87	94.88	8.00
2	58,097	3	3	0	0	0	0	1.50	0.00	10.14	0.00
3	61,000	0	28	0	0	0	0	14.00	1.43	10.93	0.00
4	58,092	0	0	0	0	0	0	0.00	1.43	3.62	0.00
5	58,824	21	0	0	0	0	0	0.00	1.40	20.58	0.00
6	58,704	97	72	98	0	0	0	107.50	0.00	11.61	0.00
7	58,936	19	19	34	0	6	0	46.50	3.90	14.53	1.00
8	59,387	3	6	15	0	0	0	29.50	0.00	3.59	0.00
9	59,208	15	0	0	0	0	0	0.00	0.00	0.00	0.00
10	110,913	50	24	56	0	0	0	70.00	0.00	0.00	0.00
11	108,717	58	15	30	0	0	2	30.00	0.00	14.45	5.00
12	108,932	0	10	41	0	0	0	43.00	0.00	1.37	0.00
13	110,086	49	27	0	0	0	0	25.50	0.00	24.67	1.00
14	108,485	62	0	0	0	0	0	25.50	0.00	0.00	0.00
15	110,845	4	4	0	0	0	0	2.00	1.33	4.78	0.00
16	106,077	2	2	30	0	0	0	41.00	1.27	0.59	0.00
17	106,686	0	1	0	0	0	0	4.50	0.00	0.00	0.00
18	109,829	4	7	41	0	4	0	74.00	1.60	6.00	1.00
19	106,723	0	0	0	0	0	0	0.00	0.00	0.00	0.00
20	151,809	19	19	30	0	0	0	40.00	0.00	2.08	0.00
21	148,772	0	0	0	0	0	0	0.00	0.00	0.00	0.00
22	151,909	9	0	0	0	0	0	0.00	0.00	0.00	0.00
23	151,324	49	43	43	0	0	0	43.00	0.00	47.61	0.00

Table 2 (continued)

ID	OPT	Best		Average							
		GA	F&F	QPSO	TPTEA	DQPSO	MFSS	QPSO	TPTEA	DQPSO	MFSS
24	151,966	18	0	0	0	0	0	28.00	4.20	12.06	0.00
25	152,109	0	0	0	0	0	0	0.00	0.00	0.00	0.00
26	153,131	0	0	0	0	0	0	0.00	0.00	0.00	0.00
27	153,578	58	45	49	0	0	0	49.00	0.00	17.60	0.00
28	149,160	5	0	0	0	0	0	15.00	0.00	3.47	0.00
29	149,704	0	16	58	0	0	0	67.00	0.00	0.00	0.00
Avg.		22.13	15.07	17.67	0.00	2.87	0.07	27.28	1.81	10.15	0.53
#worse		20	19	13	0	3		22	9	19	
#equal		10	11	17	29	26		8	19	11	
#better		0	0	0	1	1		0	2	0	

The evaluation is done based on the difference between the optimal solution and the solution acquired by a method ($OPT - X$). The columns “Best” represent the minimal difference, and “Average” is the average difference over ten independent runs

Table 3 Comparison of the state-of-the-art methods to MFSS on instances with 250 items and 30 constraints

ID	BKS	Best		Average				
		GA	F&F	QPSO	TPTEA	DQPSO	MFSS	MFSS
0	56,842	149	46	46	18	46	0	16.20
1	58,520	202	187	218	0	169	0	11.40
2	56,614	61	61	0	0	0	0	29.40
3	56,930	67	0	0	0	0	0	0.00
4	56,629	0	0	0	0	0	0	0.00
5	57,205	86	56	59	0	16	0	15.50
6	56,357	65	94	54	0	54	4	27.20
7	56,457	54	0	65	0	0	0	0.00
8	57,474	32	101	27	0	0	0	12.80
9	56,447	0	0	0	0	0	0	0.00
10	107,770	81	35	67	0	38	0	15.90
11	108,392	54	54	54	0	13	13	13.00
12	106,442	57	27	0	0	0	0	0.80
13	106,876	80	44	25	0	0	0	7.10
14	107,414	18	0	32	0	18	0	0.00
15	107,271	25	0	0	0	0	0	2.50
16	106,372	64	95	124	0	7	0	10.10
17	104,032	39	29	44	0	18	0	17.20
18	106,856	21	21	0	0	21	0	14.70
19	105,780	29	38	29	0	29	0	10.00
20	150,163	80	25	67	0	25	0	5.00
21	149,958	51	0	0	0	51	0	0.00
22	153,007	14	0	0	0	0	0	0.00
23	153,234	65	52	0	0	0	0	1.30

Table 3 (continued)

ID	BKS	Best		Average							
		GA	F&F	QPSO	TPTEA	DQPSO	MFSS	QPSO	TPTEA	DQPSO	MFSS
24	150,287	0	0	0	0	0	0	0.00	0.00	0.00	0.00
25	148,574	30	25	30	0	0	0	45.50	0.00	13.26	3.10
26	147,477	6	22	6	0	0	0	14.00	0.00	0.00	0.00
27	152,912	71	71	77	0	0	0	77.00	0.00	17.63	0.00
28	149,570	2	0	0	0	0	0	29.00	0.00	0.14	0.00
29	149,668	96	81	0	0	67	0	48.00	0.00	67.00	9.70
Avg.		53.30	38.80	34.13	0.60	19.07	0.57	54.20	2.94	40.02	7.43
#worse		27	20	17	1	13		25	3	26	
#equal		3	10	13	27	17		4	11	4	
#better		0	0	0	2	0		1	16	0	

These instances do not have proven optimal solutions but only best-known solutions (BKS). The evaluation is done based on the difference between the BKS and the solution acquired by a method ($BKS - X$). The columns "Best" represent the minimal difference, and "Average" is the average difference over ten independent runs

Table 4 Comparison of the state-of-the-art methods to MFSS on instances with 500 items and five constraints

ID	OPT	Best		Average				
		GA	F&F	QPSO	TPTEA	DQPSO	MFSS	MFSS
0	120,148	18	14	18	0	0	0	7.00
1	117,879	42	15	35	0	15	0	14.00
2	121,131	22	0	0	0	0	0	3.00
3	120,804	6	10	52	0	0	0	8.00
4	122,319	0	0	0	0	0	0	0.00
5	122,024	17	0	0	0	0	0	9.00
6	119,127	14	18	33	0	0	0	6.00
7	120,568	0	0	32	0	0	0	0.00
8	121,586	11	11	0	11	11	0	12.00
9	120,717	18	10	32	0	0	0	13.00
10	218,428	6	0	0	0	2	0	1.00
11	221,202	11	0	0	11	0	0	5.00
12	217,542	8	8	14	0	6	0	2.00
13	223,560	2	2	0	0	0	0	0.00
14	218,966	4	0	1	0	0	0	0.00
15	220,530	16	0	3	0	0	0	2.00
16	219,989	2	0	46	0	0	0	1.00
17	218,215	21	0	0	0	0	0	4.00
18	216,976	0	0	0	0	0	0	0.00
19	219,719	26	0	0	0	0	0	3.00
20	295,828	0	0	0	0	0	0	0.00
21	308,086	9	7	0	0	0	0	1.00
22	299,796	0	0	8	0	0	0	0.00
23	306,480	4	4	0	0	0	0	0.00

Table 4 (continued)

ID	OPT	Best		Average							
		GA	F&F	QPSO	TPTEA	DQPSO	MFSS	QPSO	TPTEA	DQPSO	MFSS
24	300,342	0	0	0	0	0	0	32.00	1.33	0.00	0.00
25	302,571	11	0	11	0	0	0	24.00	5.60	8.75	7.00
26	301,339	17	10	17	0	0	0	21.70	8.33	9.79	7.00
27	306,454	24	24	32	0	0	0	45.00	0.00	5.64	2.00
28	302,828	14	14	0	0	0	0	19.30	7.30	4.30	4.00
29	299,910	6	6	0	0	6	4	24.70	8.20	7.81	5.00
Avg.		10.97	5.10	11.13	0.73	1.33	0.13	33.93	9.55	7.28	3.87
#worse		24	14	14	2	5		30	22	22	
#equal		6	16	15	27	25		0	5	4	
#better		0	0	1	1	0		0	3	4	

The evaluation is done based on the difference between the optimal solution and the solution acquired by a method ($OPT - X$). The columns “Best” represent the minimal difference, and “Average” is the average difference over ten independent runs

Table 5 Comparison of the state-of-the-art methods to MFSS on instances with 500 items and 10 constraints

ID	OPT	Best		Average				
		GA	F&F	QPSO	TPTEA	DQPSO	MFSS	MFSS
0	117,821	95	87	77	20	42	12	27.00
1	119,249	110	68	72	49	43	47	65.00
2	119,215	56	21	0	56	0	4	15.00
3	118,829	27	45	54	0	16	16	19.00
4	116,530	96	59	28	74	21	21	47.00
5	119,504	50	62	102	21	34	0	37.00
6	119,827	78	63	0	52	0	17	46.00
7	118,344	56	35	35	21	24	11	30.00
8	117,815	36	34	94	14	34	34	40.00
9	119,251	126	68	0	55	39	20	45.00
10	217,377	59	59	69	26	12	0	27.00
11	219,077	55	41	0	18	14	0	17.00
12	217,847	75	50	50	0	0	50	51.00
13	216,868	66	25	0	0	25	0	8.00
14	213,873	64	62	78	59	30	14	20.00
15	215,086	73	65	0	0	24	1	27.00
16	217,940	44	60	72	14	9	9	20.00
17	219,990	41	21	41	6	6	6	11.00
18	214,382	50	36	0	19	0	27	33.00
19	220,899	66	50	72	12	34	13	32.00
20	304,387	43	43	43	0	0	0	26.00
21	302,379	47	34	38	0	21	0	20.00
22	302,417	63	9	0	1	9	1	6.00
23	300,784	41	41	0	0	0	0	36.00

Table 5 (continued)

ID	OPT	Best		Average							
		GA	F&F	QPSO	TPTEA	DQPSO	MFSS	QPSO	TPTEA	DQPSO	MFSS
24	304,374	30	17	34	0	0	7	45.50	12.87	20.25	15.00
25	301,836	106	94	0	40	70	0	48.50	95.57	83.52	35.00
26	304,952	3	41	0	0	3	3	27.50	0.00	3.00	3.00
27	296,478	41	31	41	0	19	12	46.00	22.47	33.84	19.00
28	301,359	46	28	66	0	2	2	75.00	9.73	26.76	5.00
29	307,089	75	11	87	0	0	0	125.50	0.77	20.17	5.00
Avg.		60.60	45.33	38.43	18.57	17.70	<u>10.90</u>	64.00	54.82	48.72	<u>26.23</u>
#worse		29	28	18	13	14		27	24	28	
#equal		1	2	5	7	10		0	0	1	
#better		0	0	7	10	6		3	6	1	

The evaluation is done based on the difference between the optimal solution and the solution acquired by a method ($OPT - X$). The columns “Best” represent the minimal difference, and “Average” is the average difference over ten independent runs

Table 6 Comparison of the state-of-the-art methods to MFSS on instances with 500 items and 30 constraints

ID	BKS	Best		Average							
		GA	F&F	QPSO	TPTEA	DQPSO	MFSS	QPSO	TPTEA	DQPSO	MFSS
0	116,056	188	153	65	88	104	103	150.00	158.80	250.64	122.00
1	114,810	143	92	126	41	76	76	149.00	77.00	176.16	95.00
2	116,741	80	158	29	33	0	36	98.50	121.90	160.00	68.00
3	115,354	117	156	0	41	118	41	291.50	102.40	147.34	68.00
4	116,525	172	51	90	70	153	15	146.50	160.20	228.69	56.00
5	115,741	137	7	147	7	68	0	157.50	67.00	109.89	41.00
6	114,181	229	185	194	96	220	74	244.50	143.90	251.78	105.00
7	114,348	149	82	164	70	159	39	212.50	183.60	304.83	63.00
8	115,419	172	0	0	131	100	0	148.00	197.57	248.33	68.00
9	117,116	169	105	207	4	113	12	207.00	131.63	195.15	54.00
10	218,104	109	36	36	0	61	29	36.00	34.40	95.49	54.00
11	214,648	114	22	22	3	97	3	101.50	103.07	174.56	17.00
12	215,978	124	142	139	32	95	60	139.00	79.20	136.84	88.00
13	217,910	74	48	94	0	103	35	94.00	78.67	135.03	70.00
14	215,689	93	97	145	0	88	54	145.00	86.93	174.29	60.00
15	215,919	157	135	166	79	145	52	166.00	152.77	220.92	61.00
16	215,907	135	83	118	0	36	24	122.50	49.77	133.63	32.00
17	216,542	206	124	155	0	90	54	155.00	82.27	193.41	81.00
18	217,340	50	115	123	0	50	0	129.00	35.70	95.85	27.00
19	214,739	115	76	0	0	58	0	52.50	67.70	112.22	42.00
20	301,675	48	32	32	0	32	0	40.00	33.37	46.62	20.00
21	300,055	70	73	90	0	42	0	91.50	19.27	110.76	38.00
22	305,087	92	25	49	0	32	25	49.00	6.53	93.54	38.00
23	302,032	97	50	50	17	28	28	50.00	48.40	103.36	39.00

Table 6 (continued)

ID	BKS	Best		Average							
		GA	F&F	QPSO	TPTEA	DQPSO	MFSS	QPSO	TPTEA	DQPSO	MFSS
24	304,462	58	49	116	0	58	15	116.00	34.47	68.74	44.00
25	297,012	118	94	120	13	50	0	120.00	47.03	147.85	44.00
26	303,364	131	44	77	0	4	0	77.00	28.40	110.18	31.00
27	307,007	63	99	92	8	8	8	92.00	34.50	76.36	32.00
28	303,199	142	90	30	0	37	21	30.00	30.47	99.94	39.00
29	300,596	136	125	147	0	60	56	147.00	65.77	96.24	71.00
Avg.		122.93	84.93	94.10	<u>24.43</u>	76.17	28.67	125.27	82.09	149.95	<u>55.60</u>
#worse		30	28	25	7	26		28	21	30	
#equal		0	2	2	8	3		0	0	0	
#better		0	0	3	15	1		2	9	0	

These instances do not have proven optimal solutions but only best-known solutions (BKS). The evaluation is done based on the difference between the BKS and the solution acquired by a method ($BKS - X$). The columns “Best” represent the minimal difference, and “Average” is the average difference over ten independent runs

Table 7 Average computation time needed to find the best solutions for different methods for the MKP

Items $\times$ constraints	Avg.			Std		
	TPTEA	DQPSO	MFSS	TPTEA	DQPSO	MFSS
250 $\times$ 5	94.6	7.6	155.1	109.3	5.0	77.8
250 $\times$ 10	189.5	20.6	222.4	161.1	11.5	119.3
250 $\times$ 30	166.9	87.8	239.3	153.7	56.0	103.2
500 $\times$ 5	1338.4	43.8	317.8	574.8	17.7	86.3
500 $\times$ 10	1733.5	60.1	394.0	324.7	20.3	69.5
500 $\times$ 30	1822.9	219.7	384.8	216.1	48.0	51.5

Another aspect of the performance of the MFSS to be analyzed is the computational cost, through the evaluation of the time needed to find the best solution. In Table 7, the average computational times and corresponding standard deviations are presented for TPTEA, DQPSO, and MFSS. The aggregated values are provided for each problem size. To be exact, the averages over all the runs for all the instances are presented. Since the computational experiments are performed on different hardware, the computational times for TPTEA and DQPSO are scaled by the factor of 0.5 based on the information provided by PassMark (2022). There are several observations that can be made from the results in Table 7. Firstly, the computational cost of the MFSS is not highly dependent on the number of constraints in the instance, increasing around 50% when the number of constraints increases from 5 to 30. In the case of instances with 250 items, MFSS has a roughly 50% higher computational cost than the TPTEA and a significantly higher one than DQPSO. It is interesting that the MFSS has a much better scaling than the other two methods. This is illustrated by the MFSS having a close to five times lower computational cost than TPTEA for the largest instances.

4.2 Evaluation of the MFSS for the KPFS

In this section, the computational experiments used to evaluate the MFSS for the KPFS are presented. This subsection starts with providing details of the used test instances, and later, a detailed comparison to the results from D'Ambrosio et al. (2023) is provided.

4.2.1 Instances KPFS

In this subsection, a short description of the test instances proposed in D'Ambrosio et al. (2023), that are used in the comparison, is provided. These instances vary in the number of items $|V|$, where $|V|$ takes values of 300, 500, 700, 800, or 1000. They are classified into four different scenarios, each comprising instances of three different types. The scenarios are as follows:

- **Scenario 1:** The number of forfeit sets is $l = 5|V|$. The size of each forfeit set C_i is chosen randomly in the interval $[2, \frac{|V|}{50}]$ for $i = 1, \dots, l$. The allowances for each forfeit set C_i , for $i = 1, \dots, l$ satisfy $h_i = 1$.
- **Scenario 2:** The number of forfeit sets is $l = 3|V|$. The size of each forfeit set C_i is chosen randomly in the interval $[2, \frac{|V|}{20}]$ for $i = 1, \dots, l$. The allowances for each forfeit set C_i , for $i = 1, \dots, l$ satisfy $h_i = 1$.
- **Scenario 3:** The number of forfeit sets is $l = 5|V|$. The size of each forfeit set C_i is chosen randomly in the interval $[2, \frac{|V|}{50}]$ for $i = 1, \dots, l$. The allowance h_i for forfeit set C_i is selected randomly from the interval $[1, \frac{2}{3}|C_i|]$
- **Scenario 4:** The number of forfeit sets is $l = 3|V|$. The size of each forfeit set C_i is chosen randomly in the interval $[2, \frac{|V|}{20}]$ for $i = 1, \dots, l$. The allowance h_i for forfeit set C_i is selected randomly from the interval $[1, \frac{2}{3}|C_i|]$

Scenarios 1 and 3 involve a larger number of relatively smaller forfeit sets, whereas Scenarios 2 and 4 have a smaller number of larger forfeit sets. Thus, Scenarios 1 and 3 model more specific properties related to fewer items, while the opposite is true in the other two cases. Additionally, in Scenarios 1 and 2, a single item is allowed for each set without incurring costs, making it similar to classical conflicts.

In addition to different scenarios, there are types of test instances based on relationships between profits, weights, and forfeit costs and are classified as follows:

- **Not Correlated (NC):** Each item's weight $w_j \in W$ and item profit $p_j \in P$ are chosen uniformly at random in the interval $[1, 30]$. Each forfeit cost $d_i \in D$ is chosen at random in the interval $[1, 20]$.
- **Correlated (C):** Weights and costs are chosen randomly as in the NC instances. Each profit $p_j \in P$ is equal to the weight $w_j + 10$ of the corresponding item $j \in X$.
- **Fully Correlated (FC):** Weights are chosen randomly as in previous instance types. Profits are correlated to weights, as in the C instances. Each cost d_i is computed using the following formula:

$$d_i = \left\lfloor \frac{\sum_{j \in \bar{C}_i^+} w_j}{|C_i|} \right\rfloor \quad (17)$$

where $\bar{C}_i^+$ represents the subset consisting of the $h_i + 1$ items with the highest profits in C_i .

The capacity c is calculated as $w_{\min} + \frac{w_{\max}}{10} \times \frac{|V|}{10}$, where $w_{\min}$ and $w_{\max}$ represent the minimum and maximum bounds for w_j values (in our case, 1 and 30, respectively).

For Scenarios 1-2, the upper bound k is set as $\frac{|V|}{15}, \frac{|V|}{25}, \frac{|V|}{35}, \frac{|V|}{45}$, or $\frac{|V|}{55}$, where n takes values of 300, 500, 700, 800, or 1000, rounded to the nearest integer. In Scenarios 3-4, k is uniformly $\frac{|V|}{15}$, rounded to the nearest integer, for all $|V|$ values.

4.2.2 Comparison to other methods

The same set of MFSS parameters has been used for all the test instances in the comparison for the KPFS. These values have been found empirically. The size of the initial population size N_{Pop} is 100. The number of n best solutions that are considered for selecting the base solution and S_{kn} is 100. The value of the parameter k for generating S_{kn} is a randomly selected integer value from the interval $[5, 9]$. The algorithm is considered stagnant if in the last $\text{MaxStag} = 20$ iterations, there have been no changes of the set of the n best solutions S_n . The maximal allowed calculation time for the MFSS is 600 s. As it has been discussed in D'Ambrosio et al. (2023), the performance of the IP solver highly varied depending on the type of instances. Due to this fact with the goal of achieving the best results, different values for the initial size of the fixed set are used for the MFSS. To be specific, for instances corresponding to scenarios 3 and 4, on which the IP solver is most effective, the value $\text{SizeMax} = 200$ is used for the size of the initial subproblem being solved. In the case of scenarios 1 and 2, for not correlated and correlated instances, SizeMax had the value 100. In the case of instances of the fully correlated type for scenarios 1 and 2, that are hardest to solve for the IP solver, the value of $\text{SizeMax} = 80$ is used. Finally, the value of $\delta = 0.9$ is used for the portion of acquired proven optimal solutions for which the size of the fixed set is decreased. The value $\gamma = 1.25$ is used for decreasing the size of the fixed set.

The experimental setup is the same as in D'Ambrosio et al. (2023), where a single run is performed on all of the ten instances in a group having a specific number of items $|V|$, scenario, and type. For each group of instances, the average solution value is given over the ten instances. These values for the MFSS for scenarios 1 and 2 are shown in Table 8, while Table 9 provides the same information for scenarios 3 and 4. These tables also contain the results for the memetic algorithm (MA) approach taken from D'Ambrosio et al. (2023). In addition, the comparison includes the averages solution values acquired using CPLEX with a time limit of 3 h taken from D'Ambrosio et al. (2023).

The first thing that can be observed from these results is that the MFSS manages to significantly outperform the MA in case of scenarios 3 and 4. The average solution quality and best-found solution quality is higher for 30 out of 30 instance groups. The MFSS has a better performance than MA in 11 and five instance groups, while the opposite is true for four and nine instances, for scenarios 1 and 2, respectively. The advantage of the MFSS compared to MA is the highest in the case of NC and C type instances, where it had a lower average solution value for only one and three instance groups, respectively. On the other hand, the MA managed to outperform the MFSS in case of FC instances for scenarios 1 and 2. As it is discussed by D'Ambrosio et al. (2023), CPLEX has the worst performance on these types of problems. This indicates that the performance of the MFSS is highly dependent on the effectiveness of the IPS.

The MA approach is computationally less expensive than the MFSS, it had average computational times between 1 and 120 s until stagnation occurred (no improvement to the best solutions is found in 150 iterations) for the smallest to largest instances, respectively. In case of the MFSS, the average time to finding the best

Table 8 Comparison of the state-of-the-art methods to MFSS on instances with various numbers of items, instance types, and scenarios for the KPFS

Type	Items	Scenario 1			Scenario 2		
		CPLEX	MA	MFSS	CPLEX	MA	MFSS
NC	300	684.0	681.4	684.0	<u>299.6</u>	299.1	299.6
NC	500	561.7	560.8	570.2	<u>227.6</u>	226.7	227.6
NC	700	–	492.0	499.7	<u>186.9</u>	186.3	186.9
NC	800	–	457.1	457.6	174.3	174.3	172.4
NC	1000	–	404.6	404.7	<u>145.9</u>	144.0	144.7
C	300	769.5	768.7	772.0	<u>443.9</u>	443.9	443.9
C	500	834.1	842.2	843.8	343.2	346.5	346.9
C	700	–	727.5	733.4	–	273.7	273.6
C	800	–	667.3	670.4	–	249.8	248.1
C	1000	–	598.2	593.6	–	213.7	211.0
FC	300	751.3	751.9	753.0	464.4	465.4	464.6
FC	500	802.6	819.2	812.8	403.4	406.5	404.3
FC	700	–	742.7	743.1	–	348.9	348.0
FC	800	–	706.0	702.4	–	325.9	323.6
FC	1000	–	641.9	634.1	–	289.9	285.6
Avg.			657.43	658.32		292.97	292.05
#Better			4			9	
#Equal			0			1	
#Worse			11			5	

The underlined values for CPLEX are used for proven optimal values

solution is between 60 and 500 s for the smallest to largest instances, respectively. It is important to point out that it takes the MFSS a longer time period to get trapped in locally optimal solutions than the MA. It is interesting to mention that the MA is most effective on instances of scenario 2, for which it has significantly lower computational times than for the other scenarios, between 1 and 11 s, as has been discussed by D'Ambrosio et al. (2023).

From the results presented by D'Ambrosio et al. (2023), except for scenarios 1 and 3 of type NC, CPLEX becomes highly ineffective for instances having more than 500 items. The use of CPLEX managed to find all proven optimal solutions for 13 instance groups out of which the MFSS managed to find all optimal solutions for ten instance groups.

4.3 Method parameters

In this subsection, an analysis of the effect of the parameters is provided. As in the case of the FSS (Jovanovic et al. 2019; Jovanovic and Voß 2019; Jovanovic and Voss 2020; Jovanovic and Voß 2021; Jovanovic et al. 2023b), the MFSS is highly

Table 9 Comparison of the state-of-the-art methods to MFSS on instances with various numbers of items, instance types, and scenarios for the KPFS

Type	Items	Scenario 3			Scenario 4		
		CPLEX	MA	MFSS	CPLEX	MA	MFSS
NC	300	<u>1033.3</u>	1027.7	1033.3	<u>908.7</u>	903.0	908.7
NC	500	1404.5	1386.8	1404.2	<u>1178.7</u>	1168.3	1178.7
NC	700	–	1676.0	1695.0	<u>1428.3</u>	1401.9	1427.2
NC	800	–	1813.1	1836.9	1475.4	1449.4	1474.4
NC	1000	–	1949.8	1987.0	1544.4	1513.2	1543.7
C	300	<u>968.3</u>	960.2	968.3	<u>906.2</u>	896.4	906.2
C	500	1453.8	1439.4	1456.2	1321.5	1308.1	1321.5
C	700	–	1859.0	1884.8	–	1726.4	1748.3
C	800	–	2068.7	2089.3	–	1879.3	1905.0
C	1000	–	2426.4	2449.2	–	2150.4	2178.3
FC	300	<u>955.0</u>	952.0	954.8	<u>895.8</u>	889.7	895.8
FC	500	1435.6	1423.9	1436.0	1323.4	1313.6	1324.6
FC	700	–	1843.9	1864.4	–	1759.9	1781.0
FC	800	–	2054.0	2072.1	–	1937.2	1957.2
FC	1000	–	2433.1	2453.2	–	2302.3	2328.4
Avg.			1687.60	1705.65		1506.61	1525.27
#Better			0			0	
#Equal			0			0	
#Worse			15			15	

The underlined values for CPLEX are used for proven optimal values

robust in the sense that it has a good performance for a wide range of parameter values. Keep in mind that certain parameters of the MFSS are related to the execution time of the IPS and their optimal values are hardware-dependent. Our aim is to understand the impact of the MFSS parameters without emphasizing specific values.

The initial step of the algorithm is generating the initial population of solutions (see Sarhani et al. (2023) for some general exposition on initial populations in metaheuristics). In practice, it is most important to generate at least n solutions that are used in the learning mechanism, consequently, N_{pop} should be equal or greater than n . Since a random method is used for generating solutions, which eventually produces low-quality feasible solutions, there is no significant advantage in using a higher value of N_{pop} than n . On the other hand, if a more advanced method is used for generating initial solutions, i.e., by incorporating some local search, the value of N_{pop} should be higher since there is a potential of acquiring more high-quality solutions. This is due to the fact that the use of the IPS is computationally expensive, and it is preferable to only apply it to high-quality solutions.

The parameters $k_{\min}$ and $k_{\max}$ are used to provide diversity in generating new solutions, through the randomization of the size of the set S_{kn} . A small value of k

results in fixed sets with a higher level of randomness. If a smaller number of test solutions is considered, there is a higher number of elements $e \in B$ with the same value of the $O(e, S_{kn})$; consequently, a larger number of them will be randomly removed from $\hat{F}$. On the other hand, a higher value of k makes the generation of fixed sets more deterministic since there are less such elements, and the difference between $Size$ and $|\hat{F}|$ is lower.

The value of the initial computational time t_{init} of the IPS is highly hardware-dependent and is found empirically. In general, it should be the lowest value that makes it highly probable for the IPS to improve the solutions in the initial population. In relation, *StagMax* is used to increase the computational time for the IPS and is effective for a very wide range of values. It has been observed that after a minimal value of *MaxStag* for which the solution space is effectively explored, higher values of *MaxStag* only increase the computational cost without providing additional benefits. In our initial test, we have observed that *MaxStag* = 50 and *MaxStag* = 20 are effective values for the used hardware and the MKP and KPFS, respectively.

The parameters γ and δ related to the change in the size of the fixed set have the following effect. The portion of acquired proven optimal solutions δ that is considered to result in stagnation related to the size of the fixed set had the following effect. In the case, this value is too high, very rarely will the size of the fixed set be decreased. Consequently, when the allowed computational time is increased, there will be no change in the size of the neighborhood being explored, so it is unlikely that stagnation of the algorithm will stop. On the other hand, if the value of δ is too low, the size of the fixed set will be decreased too frequently, and the resulting subproblems will quickly become of a size too large for the IPS to be solved effectively. We have empirically observed that for the KPFS, a value of $\delta = 0.9$ produces good results. The value of the parameter γ is used to decrease the size of the fixed sets being used in the MFSS. The idea is to gradually decrease the size of the fixed set avoiding prematurely generating subproblems that are too large for the IPS to effectively solve with the current quality of fixed sets and time limit. In the case of the KPFS, we have observed that the value $\gamma = 1.25$ produces good results, but we expect that the value of this method parameter is highly problem-dependent.

The size of the subproblem being solved, $Size_{max}$, has a high impact on the performance of the MFSS when both convergence speed and solution quality are considered. A graphical illustration of the effect of this parameter is shown in Fig. 3 for one instance of the MKP but similar behavior is observed for the KPFS. In it, the convergence speed of the average solution quality of ten independent runs of the MFSS for different values of $Size_{max}$ is presented. From this figure, it can be observed that a higher value of $Size_{max}$ provides an increase in convergence speed and solution quality. In practice, this means that its value should be at most one for which the IPS can be effectively applied within a short computational time. It is important to point out that similar behavior can be observed for instances with a different number of items and constraints.

The last parameter that is analyzed is the size of the number of the best solutions n that is considered when generating the base solution B and the set of test solutions S_{kn} . The main idea of the MFSS is to make it possible to solve significantly larger instances than it is possible using the IP, and the effect of the

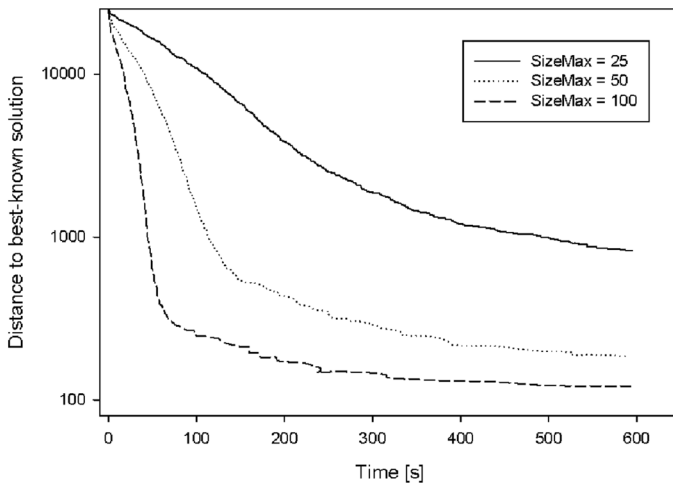


Fig. 3 Illustration of convergence speed for different values of $SizeMax$ for the instance with 500 items, 30 constraints and $ID = 0$. The graphs represent the average distance to the best-known solution over ten runs at different time periods

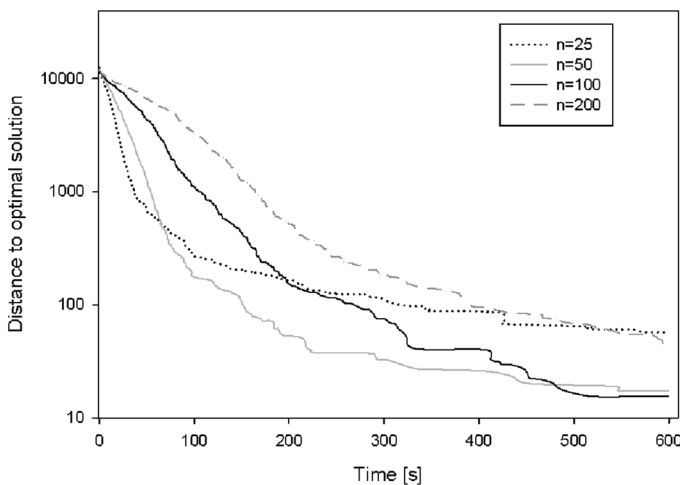


Fig. 4 Illustration of the convergence speed for different values of n for the instance with 250 items, 10 constraints and $ID = 0$. The MFSS solves subproblems with $Size_{max} = 25$ items. The graphs represent the average distance to an optimal solution over ten runs at different time periods

parameter n is observed in this context. Because of this, the effect of this parameter is evaluated for the case when the number of items (IV) in the problem instance is much larger than the number of items $Size_{max}$ for which the subproblem can be effectively solved. In Fig. 4, the average convergence speed over ten independent runs of MFSS with $Size_{max} = 25$ for a problem instance with 250 items and 10 constraints is presented. It is important to point out that similar behavior

can be observed for other instances of the MKP and for the KPFS. From Fig. 4, it can be observed that for a smaller population of test solutions, parameter n , the initial convergence speed is much higher but the method can easily get trapped in a locally optimal solution. By increasing the value of n , the convergence speed is decreased, but the MFSS has a higher probability of escaping locally optimal solutions. In practice, a larger value of n results in a wider search of the solution space. Consequently, if a long execution time is available, it is possible to acquire higher quality solutions for larger values of n .

It is important to mention once more that the performance of the MFSS is highly dependent on the used IPS. That is, to be specific, it depends on the size of problems that the IPS can effectively solve. It has been empirically observed that the MFSS is highly effective in solving problem instances 7–10 times larger than the IPS can be effectively applied to. On the other hand, standard metaheuristics can frequently be applied to much larger problem instances compared to the IPS. In practice, this means that above a certain problem size, it is more suitable to use metaheuristics than the MFSS.

5 Conclusions

In this paper, two generalizations of the knapsack problem, the multidimensional knapsack problem (MKP) and the knapsack problem with forfeit sets (KPFS), have been solved using a population-based matheuristic. To be specific, the fixed set search (FSS) has been extended to a matheuristic setting. To achieve this, a new ground set of elements for the family of knapsack problems has been introduced that maximizes the amount of information that the fixed set provides. In addition, the method for generating fixed sets has been adapted to increase the diversity of the generated solutions. The main advantage of the proposed method, compared to existing ones for the MKP and the KPFS, is the simplicity of implementation in the sense of the adaptability of the method to different problems. That is, to a large extent, due to the use of the FSS learning mechanism and by avoiding the need for defining solution neighborhoods. The computational experiments have shown that the MFSS is highly competitive with state-of-the-art methods for the problems of interest. The proposed approach is robust in the sense that it is effective for a wide range of method parameters.

It is important to point out that the MFSS does not exploit any specific properties of the MKP or the KPFS but only uses the respective IP model. This indicates the potential future application of the MFSS to other 0-1 problems, like the minimum vertex cover problem, the facility location problem, etc., without the need for large changes to the method. The practicality of the MFSS's simple adaptation to complex problems is effectively illustrated through its use for optimizing the scheduling of electric buses in public transport (Jovanovic et al. 2023a).

On the other hand, due to the simplicity of the approach, it is possible to improve its performance by hybridizing it with other heuristic/metaheuristic approaches or by exploiting some specific properties of the MKP or the KPFS.

Funding Open Access funding provided by the Qatar National Library.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Al-Shihabi S, Ólafsson S (2010) A hybrid of nested partition, binary ant system, and linear programming for the multidimensional knapsack problem. *Comput Oper Res* 37(2):247–255
- Angeles E, Mansini R, Speranza MG (2010) Kernel search: a general heuristic for the multi-dimensional knapsack problem. *Comput Oper Res* 37(11):2017–2026
- Arin A, Rabadi G (2016) Local search versus path relinking in metaheuristics: redesigning Meta-RaPS with application to the multidimensional knapsack problem. *Appl Soft Comput* 46:317–327
- Basnet C (2018) Heuristics for the multiple knapsack problem with conflicts. *Int J Oper Res* 32(4):514–525
- Boschetti MA, Maniezzo V (2022) Matheuristics: using mathematics for heuristic design. *4OR* 20(2):173–208
- Boussier S, Vazquez M, Vimont Y, Hanafi S, Michelon P (2010) A multi-level search strategy for the 0–1 multidimensional knapsack problem. *Discret Appl Math* 158(2):97–109
- Capobianco G, D'Ambrosio C, Pavone L, Raiconi A, Vitale G, Sebastiano F (2022) A hybrid metaheuristic for the knapsack problem with forfeits. *Soft Comput* 26:749–762
- Caserta M, Voß S (2019) The robust multiple-choice multidimensional knapsack problem. *Omega* 86:16–27. <https://doi.org/10.1016/j.omega.2018.06.014>
- Chen Y, Hao JK (2014) A “reduce and solve” approach for the multiple-choice multidimensional knapsack problem. *Eur J Oper Res* 239(2):313–322. <https://doi.org/10.1016/j.ejor.2014.05.025>
- Chu PC, Beasley JE (1998) A genetic algorithm for the multidimensional knapsack problem. *J Heuristics* 4:63–86
- Coniglio S, Furini F, San Segundo P (2021) A new combinatorial branch-and-bound algorithm for the knapsack problem with conflicts. *Eur J Oper Res* 289(2):435–455
- Dammeyer F, Voß S (1993) Dynamic tabu list management using the reverse elimination method. *Ann Oper Res* 41:29–46. <https://doi.org/10.1007/BF02022561>
- Drexel A (1988) A simulated annealing approach to the multiconstraint zero-one knapsack problem. *Computing* 40(1):1–8
- D'Ambrosio C, Laureana F, Raiconi A, Vitale G (2023) The knapsack problem with forfeit sets. *Comput Oper Res* 151(106):093
- Feo TA, Resende MG (1995) Greedy randomized adaptive search procedures. *J Global Optim* 6(2):109–133
- Fréville A (2004) The multidimensional 0–1 knapsack problem: an overview. *Eur J Oper Res* 155(1):1–21. [https://doi.org/10.1016/S0377-2217\(03\)00274-1](https://doi.org/10.1016/S0377-2217(03)00274-1)
- Galli L, Martello S, Rey C, Toth P (2023) Lagrangian matheuristics for the quadratic multiple knapsack problem. *Discret Appl Math* 335:36–51
- García J, Lalla-Ruiz E, Voß S, Droguett E (2020) Enhancing a machine learning binarization framework by perturbation operators: analysis on the multidimensional knapsack problem. *Int J Mach Learn Cybern* 11:1951–1970. <https://doi.org/10.1007/s13042-020-01085-8>
- Garey MR, Johnson DS (1979) *Computers and intractability: a guide to the theory of NP-completeness*. W. H. Freeman, New York
- Gilmore P, Gomory RE (1966) The theory and computation of knapsack functions. *Oper Res* 14(6):1045–1074

- Glover F, Kochenberger GA (1996) Critical event tabu search for multidimensional knapsack problems. In: Osman IH, Kelly JP (eds) *Meta-heuristics: theory and applications*. Kluwer, Boston, pp 407–427. https://doi.org/10.1007/978-1-4613-1361-8_25
- Haddar B, Khemakhem M, Hanafi S, Wilbaut C (2016) A hybrid quantum particle swarm optimization for the multidimensional knapsack problem. *Eng Appl Artif Intell* 55:1–13
- Hanafi S, Freville A (1998) An efficient tabu search approach for the 0–1 multidimensional knapsack problem. *Eur J Oper Res* 106(2–3):659–675
- Jovanovic R, Voß S (2019) Fixed set search applied to the minimum weighted vertex cover problem. In: *International symposium on experimental algorithms*. Springer, pp 490–504
- Jovanovic R, Voss S (2020) The fixed set search applied to the power dominating set problem. *Expert Syst* 37(6):e12559
- Jovanovic R, Voß S (2021) Fixed set search application for minimizing the makespan on unrelated parallel machines with sequence-dependent setup times. *Appl Soft Comput* 110(107):521
- Jovanovic R, Tuba M, Voß S (2019) Fixed set search applied to the traveling salesman problem. In: *International workshop on hybrid metaheuristics*. Springer, pp 63–77
- Jovanovic R, Sanfilippo AP, Voß S (2022) Fixed set search applied to the multi-objective minimum weighted vertex cover problem. *J Heuristics* 28:481–508
- Jovanovic R, Bayhan S, Voß S (2023a) Matheuristic fixed set search applied to electric bus fleet scheduling. In: Sellmann M, Tierney K (eds) *Learning and intelligent optimization*. Springer, Cham, pp 393–407
- Jovanovic R, Sanfilippo AP, Voß S (2023b) Fixed set search applied to the clique partitioning problem. *Eur J Oper Res* 309(1):65–81
- Khemakhem M, Haddar B, Chebil K, Hanafi S (2012) A filter-and-fan metaheuristic for the 0–1 multidimensional knapsack problem. *Int J Appl Metaheur Comput (IJAMC)* 3(4):43–63
- Lai X, Hao JK, Glover F, Lü Z (2018) A two-phase tabu-evolutionary algorithm for the 0–1 multidimensional knapsack problem. *Inf Sci* 436–437:282–301. <https://doi.org/10.1016/j.ins.2018.01.026>
- Lai X, Hao JK, Yue D (2019) Two-stage solution-based tabu search for the multidemand multidimensional knapsack problem. *Eur J Oper Res* 274(1):35–48. <https://doi.org/10.1016/j.ejor.2018.10.001>
- Lai X, Hao JK, Fu ZH, Yue D (2020) Diversity-preserving quantum particle swarm optimization for the multidimensional knapsack problem. *Expert Syst Appl* 149(113):310. <https://doi.org/10.1016/j.eswa.2020.113310>
- Li J, Lan Y, Chen F, Han X, Blazewicz J (2021) A fast algorithm for knapsack problem with conflict graph. *Asia-Pacific J Oper Res* 38(06):2150010
- Lozano-Osorio I, Sánchez-Oro J, Martínez-Gavara A, López-Sánchez AD, Duarte A (2023) An efficient fixed set search for the covering location with interconnected facilities problem. In: *Metaheuristics: 14th international conference*. Springer, pp 485–490
- Maniezzo V, Boschetti MA, Stützle T (2021) Kernel search. In: *Matheuristics*. Springer, pp 189–197
- Mancini S, Ciavotta M, Meloni C (2021) The multiple multidimensional knapsack with family-split penalties. *Eur J Oper Res* 289(3):987–998. <https://doi.org/10.1016/j.ejor.2019.07.052>
- Mansini R, Speranza MG (2012) Coral: an exact algorithm for the multidimensional knapsack problem. *Inform J Comput* 24(3):399–415
- Mingo López LF, Gómez Blas N, Arteta Albert A (2018) Multidimensional knapsack problem optimization using a binary particle swarm model with genetic operations. *Soft Comput* 22:2567–2582
- PassMark (2022) CPU benchmark. www.cpubenchmark.net, last visited 2022-01-30
- Pferschy U, Schauer J (2009) The knapsack problem with conflict graphs. *J Graph Algorithms Appl* 13(2):233–249
- Rezoug A, Bader-El-Den M, Boughaci D (2018) Guided genetic algorithm for the multidimensional knapsack problem. *Memet Comput* 10:29–42
- Rosing K, ReVelle C (1997) Heuristic concentration: two stage solution construction. *Eur J Oper Res* 97(1):75–86
- Sarhani M, Voß S, Jovanovic R (2023) Initialization of metaheuristics: comprehensive review, critical analysis, and research directions. *Int Trans Oper Res* 30:3361–3397. <https://doi.org/10.1111/itor.13237>
- Shih W (1979) A branch and bound method for the multiconstraint zero-one knapsack problem. *J Oper Res Soc* 30(4):369–378

- Sondergeld L, Voß S (1999) Cooperative intelligent search using adaptive memory techniques. In: Voß S, Martello S, Osman I, Roucairol C (eds) *Meta-heuristics: advances and trends in local search paradigms for optimization*. Kluwer, Boston, pp 297–312
- Taillard E, Voß S (2002) POPMUSIC—a partial optimization metaheuristic under special intensification conditions. In: Ribeiro C, Hansen P (eds) *Essays and surveys in metaheuristics*. Kluwer, Boston, pp 613–629
- Vasquez M, Vimont Y (2005) Improved results on the 0–1 multidimensional knapsack problem. *Eur J Oper Res* 165(1):70–81
- Vimont Y, Boussier S, Vasquez M (2008) Reduced costs propagation in an efficient implicit enumeration for the 01 multidimensional knapsack problem. *J Comb Optim* 15(2):165–178
- Voß S, Gutenschwager K (1998) A chunking based genetic algorithm for the Steiner tree problem in graphs. In: Pardalos P, Du DZ (eds) *Network design: connectivity and facilities location*, DIMACS series in discrete mathematics and theoretical computer science, vol 40. Princeton, AMS, pp 335–355
- Woodruff D (1998) Proposals for chunking and tabu search. *Eur J Oper Res* 106:585–598

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.